Computing local delegations: the complete procedure

18 October 2026

1 The problem

The problem, in the author's words:

> We have a system which is a graph of components where each component has labeled/typed inputs and outputs ("typed" means the variables are distinct and each is assigned to a known component; there are no numeric range bounds). We have a global query contract written in PolySTL (polytopic signal temporal logic: STL whose predicates are convex polytopes over the state variables, $\mu(x) := Ax \le b$, where x is the vector of ALL system variables). We need to ALGEBRAICALLY DECOMPOSE the global contract into obligations for the components. The delegate contract handed to a component may ONLY reference the variables that component has available, i.e. its own inputs and outputs. Only some components will receive a (non-trivial) delegated contract: not every global contract needs every component. All that is available is: (1) the architectural graph (components, typed/labelled ports, who reads/writes which variable), and (2) the top-level query contract. No component models.

The composition of the delegated contracts must refine the query, and the algebra may use only contract operations: refinement, composition, quotient and projection.

The solution in plain words. Relay: Radio reads r and sends m to Display, which outputs d; the query is "if $|r| \le 1$ then $|d - r| \le \frac{1}{2}$". Refinement by the composition amounts to two implications. Contracts no component can meet satisfy them, so each contract must also be met by a component computing its outputs from the inputs received so far. None is best; the procedure picks one. *Forwarding:* each component copies onto its outgoing variables what the next cannot see ($m = r$). *Choice:* one linear-arithmetic sentence says that for every input each component in turn has an output making the query true; eliminating its quantifiers gives each output a rule, the first of fixed candidates that works ($d = r$, in Display's input $d = m$). *Enlargement:* quotient and projection enlarge each contract until none can be enlarged alone. If the query mentions only ports of relevant components forming a forwarding chain, this is complete and a false sentence proves none exists; otherwise it can refute, and for $H = 0$ search without a halting guarantee (Section 10).

2 Objects

Assumptions and variables. V is a finite set of real-valued variables without range bounds. Each is written by exactly one component or by the environment (an *environment variable*). Component i reads $\mathrm{in}(i) \subseteq V$ and writes $\mathrm{out}(i) \subseteq V$; $\mathrm{ports}(i) := \mathrm{in}(i) \cup \mathrm{out}(i)$. The *architecture graph* has an edge $u \to w$ when w reads a variable that u writes. We assume: (i) discrete time $t \in \{0, \dots, H\}$; (ii) no directed cycle; (iii) zero delay: an output at time t may depend on inputs at time t. The *timed variable* $v(t)$ is the value of v at time t. A *trace* assigns a real number to every timed variable; a *history* of $W \subseteq V$ assigns one to every $v(t)$ with $v \in W$; $\overline{W}$ is the set of timed variables $v(t)$ with $v \notin W$.

The formula language $\mathcal{L}$. An *atom* is $\sum_{v \in V} a_v\, v(t) \le b$, rational a_v, b, one time t: a half-space over the vector of all variables at time t. true and false are the atoms $0 \le 0$ and $0 \le -1$. Formulas of $\mathcal{L}$ are built from atoms with $\neg, \wedge, \vee, \to$; a strict inequality is a negated atom; $e = c$ abbreviates $e \le c \wedge -e \le -c$, and $|e| \le c$ abbreviates $e \le c \wedge -e \le c$. A formula is identified with the set of traces satisfying it. It is *over* W if it mentions only $v(t)$ with $v \in W$; its truth then depends only on the history of W. $\varphi \models \psi$: every trace satisfying φ satisfies ψ; $\equiv$: both ways. $\exists v(t)$ ranges over $\mathbb{R}$; $\exists \overline{W}$ quantifies all of $\overline{W}$. $\varphi[y := e]$ replaces the timed variable y by the expression e. A formula with quantifiers is not in $\mathcal{L}$, but by Lemma 6(a) it is equivalent to one; below it always stands for that formula.

Bounded PolySTL. $\psi ::= \mu \mid \neg\psi \mid \psi \wedge \psi \mid \psi\, \mathcal{U}_{[a,b]} \psi$, with μ a polytope atom $a \cdot x \le b$ over the vector x of all variables. Its meaning at time t is $[\psi]_t \in \mathcal{L}$: $[\mu]_t$ replaces each v by $v(t)$; $[\cdot]_t$ commutes with $\neg, \wedge$; $[\psi_1 \mathcal{U}_{[a,b]} \psi_2]_t := \bigvee_{s=t+a}^{t+b} \big([\psi_2]_s \wedge \bigwedge_{t \le s' < s} [\psi_1]_{s'}\big)$; $\Box_{[a,b]}$, $\Diamond_{[a,b]}$ give $\bigwedge$, $\bigvee$ over $s \in [t+a, t+b]$. "Bounded": all times so referenced from $t = 0$ are $\le H$; a trace satisfies ψ if it satisfies $[\psi]_0$. *So $\mathcal{L}$ and bounded PolySTL define the same sets of traces*: $[\psi]_0 \in \mathcal{L}$, and an atom at time t is $[\Diamond_{[t,t]} \mu]_0$.

Contracts. A *contract* for component i is a pair (A_i, G_i) of formulas over $\mathrm{ports}(i)$: assumption and guarantee. The *query* is $P = (A_P, G_P)$, formulas over V. Its *implementation set* is $S := \neg A \vee G$, the traces on which it is not violated; an implementation is any set of traces inside S. It is *saturated* if $G \equiv S$ (the guarantee allows every trace violating the assumption). Replacing G by S changes neither implementations nor assumption, so all contracts below are saturated, written (A, S) with $\neg A \models S$; the query is (A_P, S_P), $S_P := \neg A_P \vee G_P$. The operations:

$$\begin{aligned}
&\text{refinement:} && (A,S) \preccurlyeq (A',S') \text{ iff } A' \models A \text{ and } S \models S'; \qquad \text{projection: } \rho_W(A,S) := \big(\exists\overline{W}\, A,\ \forall\overline{W}\, S\big);\\
&\text{composition:} && (A_1,S_1) \otimes (A_2,S_2) := \big((A_1 \wedge A_2) \vee \neg(S_1 \wedge S_2),\ S_1 \wedge S_2\big);\\
&\text{quotient:} && (A,S)/(A',S') := \big(A \wedge S',\ (S \wedge A') \vee \neg(A \wedge S')\big).
\end{aligned}$$

In words: C refines C' if it accepts every context (environment behaviour) C' accepts and promises at least as much; composition puts two components side by side, promising both guarantees and needing both assumptions unless a promise is broken; C/C' is what a component must meet so that, next to one meeting C', the pair meets C;

projection keeps what can be said over W. Each result is saturated (e.g. $\neg\exists\overline{W}A \equiv \forall\overline{W}\neg A \models \forall\overline{W}S$). By induction, $\bigotimes_i(A_i,S_i) = \big(\bigwedge_i A_i \vee \neg\bigwedge_i S_i,\ \bigwedge_i S_i\big)$: in the step, $\big((\bigwedge_{i<n} A_i \vee \neg\bigwedge_{i<n} S_i)\wedge A_n\big)\vee\neg\bigwedge_{i\le n} S_i \equiv \bigwedge_{i\le n} A_i \vee \neg\bigwedge_{i\le n} S_i$ since $\neg\bigwedge_{i<n} S_i \wedge A_n \models \neg\bigwedge_{i\le n} S_i$.

Lemma 1 (adjunction, localisation)**.** *(a) $C_1\otimes C_2 \preccurlyeq C$ iff $C_1 \preccurlyeq C/C_2$. (b) $\rho_W(C)$ is the $\preccurlyeq$-greatest contract over W that refines C.*

Proof. (a) With $C=(A,S)$, both sides say $A\wedge S_1\wedge S_2 \models A_1\wedge A_2\wedge S$ (left: use $\neg A\models S$; right: $\neg A_1\models S_1$). (b) For (A',S') over W: $A\models A'$ iff $\exists\overline{W}A\models A'$, and $S'\models S$ iff $S'\models\forall\overline{W}S$, because A',S' do not depend on $\overline{W}$. □

3 What "valid" means

A *delegation* is a family of contracts $C_i=(A_i,S_i)$, C_i over ports(i), one per component. It is *valid* if $\bigotimes_i C_i \preccurlyeq P$.

Proposition 2 (validity conditions)**.** *A delegation is valid iff (V1) $A_P\wedge\bigwedge_i S_i \models G_P$ and (V2) $A_P\wedge\bigwedge_i S_i\models A_j$ $(\forall j)$.*

Proof. By the formula for $\bigotimes$, validity means (i) $A_P\models\bigwedge_i A_i\vee\neg\bigwedge_i S_i$ and (ii) $\bigwedge_i S_i\models\neg A_P\vee G_P$. (ii) says every trace of $A_P\wedge\bigwedge_i S_i$ satisfies G_P: (V1). (i) says every trace of A_P satisfying $\bigwedge_i S_i$ satisfies $\bigwedge_i A_i$: (V2). □

Proposition 3 (validity alone is empty)**.** *(a) One component* $(\mathsf{true},\mathsf{false})$*, all others* $(\mathsf{true},\mathsf{true})$*, is valid. (b) If a rational trace z satisfies $A_P\wedge G_P$ and all variables of G_P are ports, $C_i := \big(\mathsf{true},\bigwedge_{v\in\text{ports}(i),t\le H} v(t)=z(v(t))\big)$ is valid.*

(Proof: (a) $\bigwedge_i S_i\equiv\mathsf{false}$; (b) a trace of $\bigwedge_i S_i$ agrees with z on the variables of G_P.) Both are useless: in (a) the component given $(\mathsf{true},\mathsf{false})$ has no implementation; in (b) each component with an input forbids all input histories but one. A component must accept whatever arrives at its inputs, which depends on which ports are inputs, a distinction no operation uses:

Definition 4. C_i is *receptive* (R) if for every history of in(i) some history of out(i) gives a trace in S_i. A *causal strategy* for i is a family of functions $f_{i,t}$, $t\le H$, from the values of in(i) at times $\le t$ to values of out(i) at time t. C_i is *causal* (K) if some causal strategy produces, from every history of in(i), a trace in S_i. A delegation is *acceptable* if it satisfies (V1), (V2), and (R), (K) for every component. (K) implies (R); (R), (K) survive enlarging S_i; (V1), (V2) survive shrinking it.

Proposition 5 (no greatest solution)**.** *In the* relay *(Radio reads the environment variable r, writes m; Display reads m, writes d; $H=0$, v for $v(0)$; $A_P=|r|\le 1$, $G_P=|d-r|\le\frac12$), D: Radio* $(\mathsf{true},m=r)$*, Display* $(\mathsf{true},d=m)$ *and D': Radio* $(\mathsf{true},m=-r)$*, Display* $(\mathsf{true},d=-m)$ *are acceptable, but no valid delegation has implementation sets containing those of both.*

Proof. In D and D', $d=r$: (V1); (V2) is trivial; each output is a function of the input at the same time: (K), (R). A delegation containing both has $S_{\text{Radio}}\supseteq(m=r\vee m=-r)$, $S_{\text{Display}}\supseteq(d=m\vee d=-m)$; the trace $r=1$, $m=-1$, $d=-1$ satisfies A_P and both, but $|d-r|=2$. □

What the algebra alone gives. By Lemma 1, if all contracts but C_i are fixed, then $\bigotimes_j C_j \preccurlyeq P$ iff $C_i\preccurlyeq\rho_{\text{ports}(i)}\big(P/\bigotimes_{j\ne i}C_j\big)$: one quotient and one projection. With all others $(\mathsf{true},\mathsf{true})$, $P/(\mathsf{true},\mathsf{true})=(A_P,S_P\vee\neg A_P)=P$: the relay gets Radio $\rho_{\{r,m\}}(P)=(|r|\le1,|r|>1)$ and Display $\rho_{\{m,d\}}(P)=(\mathsf{true},\mathsf{false})$, neither receptive. No acceptable relay delegation comes from P and constant contracts by the operations: these introduce no variable and m is not in P, so Radio's set is over $\{r\}$, Display's over $\{d\}$; by (R) the first holds for all r, the second contains some d_0, and (V1) would need $|d_0-r|\le\frac12$ on $[-1,1]$. So the algebra relates each contract to the others but supplies no starting point, and by Proposition 5 there is no best delegation for it to compute. Steps 2 to 7 therefore *select* one (the only part that is not a contract operation, checked by Proposition 2); steps 8, 9 use only quotient and projection.

4 The computational core: elimination with witnesses

Idea. With all timed variables but y fixed, each atom containing y compares y with one number, its bound; between consecutive bounds no atom changes truth, so φ is constant there. It suffices to test each bound, a point between each two, points beyond the outermost, and one point if there is none; the candidates supply these for every arrangement. Let $\varphi\in\mathcal{L}$ and $y=w(t)$. Each atom $\sum_v a_v\,v(s)\le b$ of φ with $a_w\ne0$ (so $s=t$) has the *bound* $\big(b-\sum_{v\ne w}a_v\,v(t)\big)/a_w$, the value of y at which it holds with equality; let $b_1,\dots,b_n$ be the distinct bounds. The *candidates* T are five tiers, in this order: (1) the midpoints $\frac12(b_a+b_{a'})$ of all pairs of distinct bounds, (2) the bounds b_a, (3) the bounds plus one, (4) the bounds minus one, (5) the constant 0. Within a tier: first those whose value is a constant, by increasing absolute value, a negative constant before the positive one of the same absolute value; then the non-constant ones, in increasing character-code (ASCII) order of their canonical printing (terms $c{*}v$ in alphabetical order of v, then the constant; reduced fractions; joined by `' + '`; e.g. `'1*r_0 + -3/8'` < `'1*r_0 + 3/8'` < `'2*r_0 + 0'`). A candidate equal to one met earlier is dropped. For $c\in T$ let $\pi_c := \varphi[y:=c]\wedge\bigwedge_{c'\text{ before }c}\neg\varphi[y:=c']$ (the *region* of c), and $\pi_0^* := \bigwedge_{c\in T}\neg\varphi[y:=c]$. The *witness* s_y is the rule "on π_c take c; on π_0^* take 0": given the other values, the first candidate at which φ is true. Its *defining formula* is $[\![y=s_y]\!] := \bigvee_c(\pi_c\wedge y=c)\vee(\pi_0^*\wedge y=0)$, true exactly when y equals the value of s_y, and $\varphi[y:=s_y] := \bigvee_c(\pi_c\wedge\varphi[y:=c])\vee(\pi_0^*\wedge\varphi[y:=0])$.

Lemma 6 (elimination with witnesses)**.** *(a)* $\exists y\,\varphi \equiv \bigvee_{c\in T}\varphi[y:=c]$ *and* $\forall y\,\varphi \equiv \bigwedge_{c\in T}\varphi[y:=c]$*. (b)* $\exists y\,\varphi \models \varphi[y:=s_y]$*. (c) Each value* c *of* s_y *is affine at time* t*;* $\varphi[y:=c]$*, the regions and* $[\![y=s_y]\!]$ *are in* $\mathcal{L}$ *(one time per atom).*

Proof. With the other values fixed, the bound values $\beta_1 < \cdots < \beta_r$ cut $\mathbb{R}$ into cells (points β_l, open gaps, two rays; $\mathbb{R}$ if $r=0$), on each of which φ is constant. Each cell contains a candidate: β_l; $\frac{1}{2}(\beta_l+\beta_{l+1})$; β_1-1, β_r+1; 0 if $r=0$. This gives (a) for $\exists$ and (b); for $\forall$ use $\neg\varphi$ (same atoms). (c): atoms with $y=w(t)$ mention only time t, so bounds are affine at time t. □

Fourier–Motzkin (for hand computation; witnesses always come from (b)). In a *case*, a conjunction of atoms and negated atoms, each bound is a lower bound ℓ or an upper bound u by the sign of a_w, and $\exists y\big(\gamma\wedge\bigwedge_\ell \ell\lhd_\ell y\wedge\bigwedge_u y\lhd_u u\big)\equiv \gamma\wedge\bigwedge_{\ell,u}\ell\lhd_{\ell u}u$ for γ free of y, with $\lhd_{\ell u}$ strict iff either bound is: the admissible y form an interval.

5 Forwarding

Which components matter. N is the set of components that write a variable of G_P, together with every component from which a directed path leads to one of them. Every other component gets $(\mathsf{true},\mathsf{true})$: nothing it does reaches G_P. A writer of an input of a component in N is in N. *(Scope)*: every variable of A_P or G_P is a port of some component of N. A *topological ordering* of N is a list $i_1,\dots,i_p$ of its components in which each comes after every component whose outputs it reads. For $k<p$: $\mathrm{priv}_k := \mathrm{ports}(i_k)\setminus\mathrm{ports}(i_{k+1})$, what i_k sees and i_{k+1} does not; $\mathrm{edge}_k := \mathrm{out}(i_k)\cap\mathrm{in}(i_{k+1})$, the *edge variables* from i_k to i_{k+1}.

Why one sentence with one quantifier order. The exact question asks for one causal strategy per component, each reading only its own inputs. In a sentence $\forall\exists\forall\exists\cdots$ each existential choice may depend on everything to its left, which can grant values never received and make the sentence true although none exists (Section 10). Forwarding removes the excess: if each component is followed by one it feeds (E), along an edge with at least as many variables as the first sees and the second does not (Cap), then i_k copies those values onto the edge, and i_{k+1} sees as copies all that $i_1,\dots,i_k$ saw (Lemma 8). Knowledge is *nested* along the list, so at one time one quantifier order grants each component only what it can know. As the sentence quantifies by time first, i_1's choice at time 1 follows i_2's inputs at time 0; (Info) stops i_2 reading the environment, which i_1 would be granted unseen. Earlier outputs of later components are granted too; under (Info) they are rules in values the chooser knows, which step 6 substitutes. Edge values are copies, not choices, so they are not quantified, and (Pure) keeps them out of the query. (In the relay Radio's only output is its edge m, so Radio chooses nothing.)

Definition 7 (nested ordering)**.** A topological ordering is *nested* if for every $k<p$: (E) $i_k\to i_{k+1}$ is an edge; (Cap) $|\mathrm{priv}_k|\le|\mathrm{edge}_k|$; (Pure) no edge variable occurs in A_P or G_P; (Info) $H=0$, or every variable read by i_k with $k\ge 2$ is written by some i_j, $j<k$.

(E) follows from (Cap): $i_k\in N$ writes a variable, in priv_k if i_{k+1} reads none. The proofs use (Info) only as: for $H>0$, no i_k, $k\ge 2$, reads an environment variable no earlier component reads. Fix injections $\sigma_k:\mathrm{priv}_k\to\mathrm{edge}_k$ pairing the two alphabetically sorted lists. The *forwarding formula* is $F_k := \bigwedge_{t\le H}\bigwedge_{v\in\mathrm{priv}_k}\sigma_k(v)(t)=v(t)$, over $\mathrm{ports}(i_k)$; $F_p := \mathsf{true}$; other edge variables are unconstrained (their values need have no physical meaning).

Lemma 8 (carriers)**.** *Let* $j<k$ *be largest with* $v\in\mathrm{ports}(i_j)$*. Put* $w_j := v$ *and, for* $j\le l<k$*,* $w_{l+1} := w_l$ *if* $w_l\in\mathrm{ports}(i_{l+1})$*, else* $w_{l+1}:=\sigma_l(w_l)$*. Then* $\kappa_k(v):=w_k\in\mathrm{in}(i_k)$ *and* $F_j\wedge\cdots\wedge F_{k-1}\models\kappa_k(v)(t)=v(t)$ *for all* t*.*

Proof. By induction $w_l\in\mathrm{ports}(i_l)$; if $w_l\notin\mathrm{ports}(i_{l+1})$ then $w_l\in\mathrm{priv}_l$, σ_l is defined, and $F_l\models w_{l+1}(t)=w_l(t)$. So $w_k\in\mathrm{ports}(i_{k-1})\cap\mathrm{ports}(i_k)$ or $w_k\in\mathrm{edge}_{k-1}$; an output of i_k is not a port of the earlier i_{k-1}, so $w_k\in\mathrm{in}(i_k)$. □

So every value an earlier component saw arrives as a copy on an input $\kappa_k(v)$ of i_k. The *decoding* $\delta_k(\varphi)$ replaces every $v(s)$ with v a port of some i_j, $j<k$, and $v\notin\mathrm{ports}(i_k)$, by $\kappa_k(v)(s)$. On traces satisfying $F_1,\dots,F_{k-1}$, φ and $\delta_k(\varphi)$ have the same truth value; δ_k keeps one time per atom.

6 The procedure

Input: the architecture graph and P. Output: an acceptable delegation, "none exists", or "does not apply".

1. *Relevance.* Write A_P, G_P in $\mathcal{L}$. Compute N; outside N: $(\mathsf{true},\mathsf{true})$. If (Scope) fails, stop: does not apply.
2. *Ordering.* Enumerate the permutations of N, sorted by name, in lexicographic order; take the first that is topological and nested. If none, stop: does not apply (Section 10 still offers a refutation test).
3. *Sentence.* $E_k(t)$ is the set of timed variables $v(t)$ with $v\in\mathrm{in}(i_k)$ not a port of any i_j, $j<k$ (*fresh inputs*; a writer of v would be in N before i_k, so v is an environment variable; under (Info) $E_k(t)=\emptyset$ for $k\ge 2$ if $H>0$). $Y_k(t)$: the $v(t)$ with $v\in\mathrm{out}(i_k)\setminus\mathrm{edge}_k$ (*terminal outputs*: not on the edge to i_{k+1}; $\mathrm{edge}_p:=\emptyset$). The *prefix* lists these timed variables by t, then k, each set in port declaration order. Let
$$\Phi := \forall E_1(0)\,\exists Y_1(0)\,\forall E_2(0)\,\exists Y_2(0)\cdots\forall E_p(H)\,\exists Y_p(H)\ (A_P\to G_P).$$
Φ is a sentence: a port of i_k is an output (in Y_k, or in edge_k, excluded by (Pure)), a fresh input, or a port of an earlier component, so by (Scope) all of $A_P\to G_P$ is quantified.
4. *Elimination (Lemma 6).* $\psi := A_P\to G_P$. For the pairs (t,k) in reverse order: for each $y\in Y_k(t)$, last first, record the witness s_y of y in ψ and replace ψ by $\exists y\,\psi$; then eliminate $\forall$ for each timed variable of $E_k(t)$. Later variables

are already eliminated, so s_y mentions only earlier ones. The final ψ is the truth value of Φ.

5. *Decision.* If Φ is false, stop: none exists (Theorem 9).
6. *Substitution.* For $y \in Y_k(t)$, take s_y as its list of pairs (region, value). If $z(s)$ in it is a terminal output of some i_j, $j > k$, then $s < t$ ($z(s)$ precedes y), so $z(s)$ occurs only in regions (values are at time t): replace each atom α mentioning it by $\bigvee_{(\pi',c')}(\pi' \wedge \alpha[z(s) := c'])$ over the pairs of $s_{z(s)}$. Repeat; each round brings in only variables before $z(s)$, so it ends with $\tilde{s}_y$.
7. *Seed.* $C_{i_k} := \big(\mathsf{true},\ F_k \wedge \bigwedge_{t \le H} \bigwedge_{y \in Y_k(t)} \delta_k[\![y = \tilde{s}_y]\!]\big)$.
8. *Sweep: quotient and projection.* For $k = 1, \dots, p$, with current sets, $R_k := A_P \wedge \bigwedge_{j \neq k} S_{i_j}$ (outside N: true), $X_k := \overline{\mathrm{ports}(i_k)}$, $S_{i_k} := \forall X_k(R_k \to G_P)$, the implementation set of $\rho_{\mathrm{ports}(i_k)}\big(P/\bigotimes_{j \neq k}(\mathsf{true}, S_{i_j})\big)$: the quotient's guarantee is $S_P \vee \neg R_k = \neg A_P \vee G_P \vee \neg R_k \equiv R_k \to G_P$, as $\neg A_P \models \neg R_k$.
9. *Assumptions: projection.* With the final sets, $A_{i_k} := \exists X_k\, R_k$, the projected quotient's assumption. Output (A_{i_k}, S_{i_k}).

Steps 4, 8, 9 may use any equivalent formula of $\mathcal{L}$; the theorems do not depend on it, the witnesses do. The script uses Z3's `qe2` output, simplified and checked equivalent (not checked for one time per atom), and reads each witness from Z3's simplified ψ, which `DELEGATE_TRACE=1` prints with T.

7 Correctness

Theorem 9 (necessity)**.** *Under (Scope), for Φ from a topological ordering with (Pure): if an acceptable delegation exists, Φ is true.*

Proof. Φ is true iff functions choosing each existential timed variable from the earlier ones make $A_P \to G_P$ hold for all universal values. By (K) each i has a causal strategy f_i with traces in S_i; choose $Y_k(t)$ as the output of f_{i_k} at t. This uses only earlier values: an input of i_k is an environment variable, in E_j for the first i_j ($j \le k$) reading it, or an output of some i_j, $j < k$; its values at times $\le t$ are quantified earlier, or are edge values that f_{i_j} computes from such values. Complete the trace (other environment values arbitrary, components outside N by their strategies): it lies in every S_i, so by (V1) it satisfies $A_P \to G_P$, which by (Scope), (Pure) mentions only quantified values. □

Theorem 10 (soundness)**.** *If the ordering is nested and Φ is true, the seed of step 7 satisfies (V1), (V2), (R), (K), and each guarantee is a formula of $\mathcal{L}$ over* $\mathrm{ports}(i_k)$.

Proof. Language. $[\![y = \tilde{s}_y]\!] \in \mathcal{L}$ by Lemma 6(c) and step 6, and δ_k keeps this. *Ports.* A timed variable of $\tilde{s}_y$, $y \in Y_k(t)$, precedes y, is not a terminal output of a later i_j, and lies in $E_j(s)$ or $Y_j(s)$ with (s, j) not after (t, k). If $j > k$ then $s < t$, so $H > 0$ and $E_j(s) = \emptyset$, while $Y_j(s)$ was removed in step 6. So it is a port of some i_j, $j \le k$, at a time $\le t$, which δ_k maps into $\mathrm{ports}(i_k)$. *(R), (K).* At time t, i_k computes $Y_k(t)$ in prefix order from $\delta_k(\tilde{s}_y)$, from its inputs at times $\le t$ (Lemma 8) and its outputs already computed, then sets $\sigma_k(v)(t) := v(t)$, other edge values 0: a causal strategy meeting the guarantee. *(V2)*: all assumptions are true. *(V1).* If x satisfies A_P and every guarantee, it satisfies $F_1, \dots, F_{p-1}$, so $x(y) = \tilde{s}_y(x)$ by decoding, and $x(y) = s_y(x)$ by induction along the prefix (step 6 replaces $z(s)$, earlier, by $s_{z(s)}$, worth $x(z(s))$ at x). With $q_1, \dots, q_n$ the prefix, $\psi_n := A_P \to G_P$ and ψ_{l-1} the elimination of q_l from ψ_l ($\psi_0 \equiv \mathsf{true}$), induction gives $x \models \psi_l$: if q_l is universal, $\psi_{l-1} \equiv \forall q_l \psi_l$; if existential, $\psi_{l-1} \models \psi_l[q_l := s_{q_l}]$ and $s_{q_l}(x) = x(q_l)$. So $x \models G_P$. □

Theorem 11 (sweep)**.** *If the seed satisfies (V1), (R), (K), steps 8 and 9 output an acceptable delegation in $\mathcal{L}$ whose sets contain the seed's. It is maximal: no formula over* $\mathrm{ports}(i_k)$ *strictly weaker than S_{i_k} satisfies (V1) with the other final sets, nor does any family weaker in every set and strictly in one. With the final sets, $(A_{i_k}, S_{i_k}) = \rho_{\mathrm{ports}(i_k)}\big(P/\bigotimes_{j \neq k}(\mathsf{true}, S_{i_j})\big) = \rho_{\mathrm{ports}(i_k)}\big(P/\bigotimes_{j \neq k} C_{i_j}\big)$, the C_{i_j} the output contracts.*

Proof. Fix k and the other sets; $X := X_k$. For S over $\mathrm{ports}(i_k)$: $R_k \wedge S \models G_P$ iff $S \models \forall X(R_k \to G_P)$ (if $x \models S$, so does every trace differing from x only on X). So step 8 assigns the weakest S satisfying (V1) with the current others; the previous one did, so sets only grow and inherit (R), (K). Let S^* be the weakest for the final others: the final family satisfies (V1), so $S_{i_k} \models S^*$; after step k the others only grew, so R_k grew and $S^* \models S_{i_k}$. A family weaker everywhere satisfying (V1) still does after restoring all but one set, so each set is $\equiv$ the final one. Step 9: $A_P \wedge \bigwedge_j S_{i_j} \models R_k \models \exists X R_k = A_{i_k}$: (V2). If $x \models \neg A_{i_k}$, no trace agreeing with x on $\mathrm{ports}(i_k)$ satisfies the final R_k, nor step 8's smaller one, so $x \models S_{i_k}$: saturated. As $S_{i_k} \equiv S^*$, the first equality is steps 8, 9; the second quotient has the same assumption and a guarantee $R_k \to (G_P \wedge \bigwedge_{j \neq k} A_{i_j})$, equivalent by Lemma 1 as the output is valid. □

So, on a nested ordering under (Scope), an acceptable delegation exists iff Φ is true, and steps 6 to 9 compute one: maximal, not greatest, and dependent on the orders of T, the prefix and the sweep.

8 The relay, every step

1. $N = \{\text{Radio, Display}\}$ (Display writes d; Radio→Display); (Scope) holds. **2.** Radio, Display is the topological ordering. $\mathrm{priv}_1 = \{r, m\} \setminus \{m, d\} = \{r\}$, $\mathrm{edge}_1 = \{m\}$; (E), (Cap) $1 \le 1$, (Pure), (Info) hold; $\sigma_1(r) = m$, $F_1 = (m = r)$. **3.** $E_1 = \{r\}$, $Y_1 = \emptyset$, $E_2 = \emptyset$, $Y_2 = \{d\}$; $\Phi = \forall r\, \exists d\, (|r| \le 1 \to |d - r| \le \frac{1}{2})$. **4.** Pair $(0, 2)$: the bounds of d are $r \pm \frac{1}{2}$; T is r, $r - \frac{1}{2}$, $r + \frac{1}{2}$, $r + \frac{3}{2}$, $r - \frac{3}{2}$, 0 ($r - \frac{1}{2} + 1$ and $r + \frac{1}{2} - 1$ are duplicates); $\psi[d := r] \equiv \mathsf{true}$, so $\pi_r \equiv \mathsf{true}$, $s_d = r$,

$\exists d\,\psi \equiv \mathsf{true}$. Pair $(0,1)$: $\forall r\,\mathsf{true} \equiv \mathsf{true}$. **5.** Φ is true. **6.** Nothing to replace. **7.** $[\![d = s_d]\!] \equiv (d = r)$ (as $\pi_r \equiv \mathsf{true}$) and $\kappa_2(r) = m$, so δ_2 gives $d = m$. Seed: Radio $(\mathsf{true}, m = r)$, Display $(\mathsf{true}, d = m)$: D of Proposition 5.
8, Radio. $X_1 = \{d\}$; $S_{\text{Radio}} := \forall d\,\big((|r| \le 1 \wedge d = m) \to |d - r| \le \frac{1}{2}\big) \equiv \big(|r| \le 1 \to |m - r| \le \frac{1}{2}\big)$, the premise holding only for $d = m$. **8, Display.** $X_2 = \{r\}$; $R_2 \equiv |r| \le 1 \wedge |m - r| \le \frac{1}{2}$; $S_{\text{Display}} := \forall r(R_2 \to |d - r| \le \frac{1}{2}) \equiv \neg\exists r\big(R_2 \wedge (r > d + \frac{1}{2} \vee r < d - \frac{1}{2})\big)$. Case $r > d + \frac{1}{2}$: lower bounds -1, $m - \frac{1}{2}$, $d + \frac{1}{2}$ (strict), upper bounds 1, $m + \frac{1}{2}$; Fourier–Motzkin: $-1 \le 1$, $-1 \le m + \frac{1}{2}$, $m - \frac{1}{2} \le 1$, $m - \frac{1}{2} \le m + \frac{1}{2}$, $d + \frac{1}{2} < 1$, $d + \frac{1}{2} < m + \frac{1}{2}$, i.e. $|m| \le \frac{3}{2} \wedge d < \frac{1}{2} \wedge d < m$. Case $r < d - \frac{1}{2}$: lower -1, $m - \frac{1}{2}$; upper 1, $m + \frac{1}{2}$, $d - \frac{1}{2}$ (strict); the new pairs give $d > -\frac{1}{2}$, $d > m$: $|m| \le \frac{3}{2} \wedge d > -\frac{1}{2} \wedge d > m$. Negating,

$$S_{\text{Display}} \equiv |m| \le \tfrac{3}{2} \to \big((d \ge \tfrac{1}{2} \vee d \ge m) \wedge (d \le -\tfrac{1}{2} \vee d \le m)\big) \quad (d = m \text{ on } |m| \le \tfrac{1}{2}).$$

9. $A_{\text{Display}} := \exists r(|r| \le 1 \wedge |m - r| \le \frac{1}{2})$: lower -1, $m - \frac{1}{2}$, upper 1, $m + \frac{1}{2}$, so $|m| \le \frac{3}{2}$. $A_{\text{Radio}} := \exists d(|r| \le 1 \wedge S_{\text{Display}}) \equiv |r| \le 1$, as $d := m$ satisfies S_{Display} for every m. Output: Radio $\big(|r| \le 1,\ |r| \le 1 \to |m - r| \le \frac{1}{2}\big)$, Display $\big(|m| \le \frac{3}{2},\ S_{\text{Display}}\big)$. **False Φ.** $H = 2$, $A_P = \Box_{[0,2]}|r| \le 1$, $G_P = \Box_{[0,2]}|d - r| \le \frac{1}{2} \wedge \Diamond_{[0,2]} d \ge \frac{1}{4}$, Φ is false ($r \equiv -1$ forces $d \le -\frac{1}{2}$).

9 Reproduction

Install with `uv venv`, then `uv pip install -p .venv/bin/python z3-solver==5.3.1.0`; from the root run `.venv/bin/python scripts/delegate.py`. It prints the Z3 version, a block per instance and regression instances (tests with known verdicts), and exits 0 iff all verdicts are as expected. "receptive & causal" is $\forall\text{in}(0)\exists\text{out}(0)\cdots\forall\text{in}(H)\exists\text{out}(H)\,S_i$ (values at time t of in(i), out(i)): (K), read through functions picking each $\exists$-value from earlier ones (Lemma 6(b)). A seed prints as `y == If(...)`, i.e. $[\![y = s_y]\!]$; the sweep's `G` is S_{i_k}, a Boolean combination of atoms printed as Z3 simplifies it (order may vary with Z3's version). First 21 lines:

```
z3 version 5.3.1
================================================================================
relay, H = 0
  A_P = And(r_0 <= 1, r_0 >= -1)
  G_P = And(d_0 + -1*r_0 <= 1/2, -1*d_0 + r_0 <= 1/2)
  N = ['Display', 'Radio']
  nested ordering: ['Radio', 'Display']
  priv(Radio) = ['r'], edge(Radio) = ['m'], forwarding {'r': 'm'}
  fresh inputs E_k: {'Radio': ['r'], 'Display': []}  terminal outputs Y_k: {'Radio': [], 'Display': ['d']}
  Phi (linearised sentence) = True
  contract Radio: A = true ; G = m_0 == r_0
  contract Display: A = true ; G = d_0 == m_0
  (V1) A_P & all G_i -> G_P : True
  Radio receptive & causal: True
  Display receptive & causal: True
  sweep to a maximal delegation (order ['Radio', 'Display'] ):
    Radio: A = And(r_0 <= 1, r_0 >= -1) ; G = Or(And(m_0 + -1*r_0 <= 1/2, -1*m_0 + r_0 <= 1/2),    Not(And(r_0 <= 1,
    ↪  r_0 >= -1)))
    Display: A = And(m_0 <= 3/2, m_0 >= -3/2) ; G = And(Or(0 <= -1*m_0 + d_0,        Not(m_0 >= -3/2),        1/2 <=
    ↪  d_0,        Not(m_0 <= 3/2)),     Or(Not(m_0 >= -3/2),        Not(m_0 <= 3/2),        0 <= m_0 + -1*d_0,
    ↪  d_0 <= -1/2))
  (V1) after sweep: True
  (V2) after sweep: True
  receptive & causal after sweep (every component): True
```

10 Scope and limits

Forks. Refutation still works: under (Scope), Φ°, built as in step 3 from any topological ordering but with $Y_k(t)$ all outputs of i_k at t, is true if an acceptable delegation exists (proof of Theorem 9, without (Pure)); so a refutation test decides Φ° for every topological ordering. A true sentence proves nothing: let unconnected Left, Right map $s_1 \mapsto a$, $s_2 \mapsto b$, $A_P = \mathsf{true}$, $G_P = |a - b| \le \frac{1}{2} \wedge |b - s_2| \le \frac{1}{2}$. For the ordering Right, Left, (Pure) holds and $\forall s_2\exists b\forall s_1\exists a\,G_P$ is true ($b = s_2$, $a = b$), but strategies $a = f(s_1)$, $b = g(s_2)$ would make g bounded (fix s_1) and unbounded: none exists ((Cap) fails; Left, Right refutes). *Remark (semi-decision, $H = 0$).* Enumerate pairs (B, rational θ): f_i^θ lists at most B pairs (region, affine map of in(i)), each region a conjunction of at most B atoms or negated atoms; the first region that holds gives the value (default 0). Substitute in topological order and decide $\forall(A_P \to G_P)[\text{out}(i) := f_i^\theta(\text{in}(i))]$ by Lemma 6; if true, output the acceptable $(\mathsf{true}, [\![y = f_i^\theta]\!])$. If an acceptable delegation exists, (K) and Lemma 6(b) give such rational strategies (disjoint regions in disjunctive normal form), so it halts; else it runs forever; no bound on B is known. The script runs one instance of this search, not the enumeration: on a fan-in graph Z3 decides the nonlinear $\exists\theta\forall(\text{environment})(A_P \to G_P)$ for one affine map per output on the whole space (60 s timeout), then checks θ linearly.
Other limits. (Scope) makes Φ a sentence; if it fails, step 1 says "does not apply". Failure of (Pure) and unit delays are not covered: a delay needs $m(t + 1) = r(t)$, not definable in $\mathcal{L}$ (with other values fixed, a formula gives a finite union of products in the $(m(t+1), r(t))$-plane, and a product inside the line is a point). **Cost.** Lemma 6(a) multiplies the n atoms by at most $n(n-1)/2 + 3n + 1$ per elimination; step 2 may try $p!$ orders.